1. Linked List to String Challenges

- Write a C program that creates a doubly linked list converts a doubly linked list into a string and returns it.
- *Test Data and Expected Output* :
- Linked List: Convert a Doubly Linked list into a string
- ----------
- Input the number of nodes: 3
- Input data for node 1 : 10
- Input data for node 2 : 20
- Input data for node 3 : 30

- Return data entered in the list as a string:
- The linked list: 10 20 30

```c
#include <stdio.h>
#include <stdlib.h>

// Structure for a doubly linked list node
struct node {
    int num;
    struct node * preptr; // Pointer to the previous node
    struct node * nextptr; // Pointer to the next node
}*stnode, *ennode; // Pointers for the starting and ending nodes

// Function prototypes
void DlListcreation(int n);
void displayDlList();

int main() {
    int n;
    stnode = NULL; // Initializing starting node to NULL
    ennode = NULL; // Initializing ending node to NULL

    printf("\n\n Doubly Linked List : Create and display a doubly linked list :\n");
    printf("-------------------------------------------------------------------\n");

    printf(" Input the number of nodes : ");
    scanf("%d", &n);

    DlListcreation(n); // Function call to create a doubly linked list
    displayDlList(); // Function call to display the doubly linked list

    return 0;
}

```

```c
// Function to create a doubly linked list
void DlListcreation(int n) {
    int i, num;
    struct node *fnNode;

    if(n >= 1) {
        stnode = (struct node *)malloc(sizeof(struct node));

        if(stnode != NULL) {
            printf(" Input data for node 1 : "); // assigning data in the first node
            scanf("%d", &num);

            stnode->num = num;
            stnode->preptr = NULL;
            stnode->nextptr = NULL;
            ennode = stnode; // Setting ending node as starting node initially

```

```
            // Loop to get input data for rest of the nodes
            for(i = 2; i <= n; i++) {
                fnNode = (struct node *)malloc(sizeof(struct node));
                if(fnNode != NULL) {
                    printf(" Input data for node %d : ", i);
                    scanf("%d", &num);
                    fnNode->num = num;
                    fnNode->preptr = ennode; // New node is linking with the previous node
                    fnNode->nextptr = NULL;

                    ennode->nextptr = fnNode; // Previous node is linking with the new node
                    ennode = fnNode; // Assign new node as last node
                } else {
                    printf(" Memory can not be allocated.");
                    break;
                }
            }
        } else {
            printf(" Memory can not be allocated.");
        }
    }
}
```

```c
// Function to display the doubly linked list
void displayDlList() {
    struct node * tmp;
    int node_num = 1; // Variable to count node numbers

    if(stnode == NULL) {
        printf(" No data found in the List yet.");
    } else {
        tmp = stnode;
        printf("\n\n Data entered on the list are :\n");

        while(tmp != NULL) {
            printf(" node %d : %d\n", node_num, tmp->num);
            node_num++;
            tmp = tmp->nextptr; // Current pointer moves to the next node
        }
    }
}
```

2. Linked List to Array Variants

- Write a C program that converts a singly linked list into an array and returns it.
- *Test Data and Expected Output* :
- Linked List: Convert a Singly Linked list into a string
- ----------
- Input the number of nodes: 3
- Input data for node 1 : 10
- Input data for node 2 : 20
- Input data for node 3 : 30

- Return data entered in the list as a string:
- The linked list: 10 20 30

```c
#include<stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for a node in a singly linked list
struct node
{
    int num;                // Data of the node
    struct node *nextptr;   // Address of the next node
} *stnode;                  // Pointer to the start of the linked list

// Function prototypes
void createNodeList(int n);
void To_Array(struct node *head, int *array);

```

```c
int main()
{
    int n;
    printf("\nLinked List: Convert a Singly Linked list into an array:\n");
    printf("-------------------------------------------------------------\n");
    printf("Input the number of nodes: ");
    scanf("%d", &n);

    // Creating a singly linked list
    createNodeList(n);

    // Converting the linked list to an array
    printf("\nReturn data entered in the list as an array:\n");
    int arr[n];
    To_Array(stnode, arr);

    // Displaying the elements of the array
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}

```

```c
// Function to create a singly linked list with 'n' nodes
void createNodeList(int n)
{
    struct node *fnNode, *tmp;
    int num, i;

    // Allocating memory for the first node
    stnode = (struct node *)malloc(sizeof(struct node));

    if(stnode == NULL) // Check if memory allocation failed
    {
        printf(" Memory can not be allocated.");
    }
    else
    {
        // Input data for the first node
        printf(" Input data for node 1 : ");
        scanf("%d", &num);
        stnode->num = num;
        stnode->nextptr = NULL;
        tmp = stnode;

```

```
61          // Creating 'n' nodes and adding to the linked list
62          for(i = 2; i <= n; i++)
63          {
64              fnNode = (struct node *)malloc(sizeof(struct node));
65              if(fnNode == NULL)
66              {
67                  printf(" Memory can not be allocated.");
68                  break;
69              }
70              else
71              {
72                  // Input data for each node
73                  printf(" Input data for node %d : ", i);
74                  scanf(" %d", &num);
75
76                  fnNode->num = num;
77                  fnNode->nextptr = NULL;
78
79                  tmp->nextptr = fnNode;
80                  tmp = tmp->nextptr;
81              }
82          }
83      }
84  }
85
```

```c
// Function to convert a singly linked list into an array
void To_Array(struct node *head, int *array) {
    struct node *current = head;
    int i = 0;
    // Traverse the linked list and copy elements to the array
    while (current != NULL) {
        array[i] = current->num;
        current = current->nextptr;
        i++;
    }
}
```

3. Merge Sorted Lists Challenges

- Write a C program to merge two sorted singly linked lists into a single sorted linked list.
- *Test Data and Expected Output* :
- Two sorted singly linked lists:
- 1 3 5 7
- 2 4 6

- After merging the said two sorted lists:
- 1 2 3 4 5 6 7

```c
 #include<stdio.h>
#include <stdlib.h>

// Node structure for the linked list
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* new_Node(int data) {
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->next = NULL;
    return node;
}

```

```
// Function to merge two sorted linked lists into a single sorted linked list
struct Node* mergeTwoLists(struct Node* head1, struct Node* head2) {
    // If either of the linked lists is empty, return the other list
    if (!head1) return head2;
    if (!head2) return head1;

    // Create a dummy node to store the result
    struct Node dummy;
    struct Node* tail = &dummy;

    // Traverse both linked lists, adding smaller elements to the result list
    while (head1 && head2) {
        if (head1->data < head2->data) {
            tail->next = head1;
            head1 = head1->next;
        } else {
            tail->next = head2;
            head2 = head2->next;
        }
        tail = tail->next;
    }

    // If either of the linked lists is not fully processed, append the rest of the elements
    tail->next = head1 ? head1 : head2;

    // Return the result
    return dummy.next;
}
```

```c
// Function to display a linked list
void displayList(struct Node* head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

// Main function to demonstrate merging of two sorted linked lists
int main() {
    // Creating two sorted singly linked lists
    struct Node* list1 = new_Node(1);
    list1->next = new_Node(3);
    list1->next->next = new_Node(5);
    list1->next->next->next = new_Node(7);

    struct Node* list2 = new_Node(2);
    list2->next = new_Node(4);
    list2->next->next = new_Node(6);

    // Displaying the two sorted lists
    printf("Two sorted singly linked lists:\n");
    displayList(list1);
    displayList(list2);

    // Merging the two lists and displaying the merged list
    struct Node* result = mergeTwoLists(list1, list2);
    printf("\nAfter merging the said two sorted lists:\n");
    displayList(result);

    return 0;
}
```

4. Duplicate Removal Challenges

- Write a C program to remove duplicates from a single sorted linked list.
- *Test Data and Expected Output* :
- Original Singly List:
- 1 2 3 3 4
- After removing duplicate elements from the said singly list:
- 1 2 3 4

- Original Singly List:
- 1 2 3 3 4 4
- After removing duplicate elements from the said singly list:
- 1 2 3 4

```c
#include<stdio.h>
#include <stdlib.h>

// Define a structure for a Node in a singly linked list
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node with given data
struct Node* newNode(int data) {
    // Allocate memory for a new node
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    // Set the data for the new node
    node->data = data;
    // Set the next pointer of the new node to NULL
    node->next = NULL;
    return node;
}

```

```c
// Function to remove duplicates from a sorted linked list
void remove_Duplicates(struct Node* head) {
    // Declare pointers to traverse the list
    struct Node *current = head, *next_next;

    // Traverse the list until the end is reached
    while (current->next != NULL) {
        // Check if the current node's value is equal to the next node's value
        if (current->data == current->next->data) {
            // Update pointers to remove the duplicate node
            next_next = current->next->next;
            free(current->next); // Free memory of the duplicate node
            current->next = next_next; // Update the link to bypass the duplicate node
        } else {
            current = current->next; // Move to the next node
        }
    }
}

// Function to display the elements of a linked list
void displayList(struct Node* head) {
    // Traverse the list and print each element
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}
```

```
49
50  // Main function where the execution starts
51  int main() {
52      // Create the first linked list
53      struct Node* head1 = newNode(1);
54      head1->next = newNode(2);
55      head1->next->next = newNode(3);
56      head1->next->next->next = newNode(3);
57      head1->next->next->next->next = newNode(4);
58
59      printf("Original Singly List:\n");
60      displayList(head1); // Display the original list
61
62      printf("After removing duplicate elements from the said singly list:\n");
63      remove_Duplicates(head1); // Remove duplicates
64      displayList(head1); // Display the modified list without duplicates
65
66      // Create the second linked list
67      struct Node* head2 = newNode(1);
68      head2->next = newNode(2);
69      head2->next->next = newNode(3);
70      head2->next->next->next = newNode(3);
71      head2->next->next->next->next = newNode(4);
72      head2->next->next->next->next->next = newNode(4);
73
```

```
73
74      printf("\nOriginal Singly List:\n");
75      displayList(head2); // Display the original list
76
77      printf("After removing duplicate elements from the said singly list:\n");
78      remove_Duplicates(head2); // Remove duplicates
79      displayList(head2); // Display the modified list without duplicates
80
81      return 0; // Indicates successful completion of the program
82  }
```